FICHE APPRENANT · PROJET SUIVI D'INCIDENTS BACKBONE

Semaine 5

Le cœur de l'application — les incidents

Durée : 5 jours Form Request Règles métier Dates Événements Eloquent

Objectifs de la semaine

- isoler la validation dans une **Form Request** ;
- traduire des **règles métier** en code (et savoir où les placer) ;
- manipuler des dates avec **Carbon** ;
- comprendre la faille de **mass assignment** et s'en protéger.

1. Les Form Requests

Quand la validation grossit, on la sort du contrôleur :

```
php artisan make:request IncidentRequest
```

```
class IncidentRequest extends FormRequest
{
    public function rules(): array
    {
        return [
            'titre' => ['required', 'string', 'max:255'],
            'severite' => ['required', Rule::enum(Severite::class)],
            'site_id' => ['required', 'exists:sites,id'],
            'debut_at' => ['required', 'date'],
            'resolu_at' => ['nullable', 'date', 'after_or_equal:debut_at'],
            'cause_racine' => ['nullable', 'string', 'required_if:statut,resolu,clos'],
        ];
    }
}
```

Le contrôleur devient très court :

```
public function store(IncidentRequest $request): RedirectResponse
{
    $incident = Incident::create($request->validated() + ['declare_par_id' => $request->user()->id]);
    // ...
}
```

- `Rule::enum(Severite::class)` n'accepte que les valeurs de l'enum.

- `required_if:statut,resolu,clos` : obligatoire **seulement si** le statut est résolu ou clos.
- `$request->validated()` ne renvoie **que** les champs validés.

2. Mass assignment : pourquoi `declare_par_id` n'est pas dans le formulaire

Si on faisait `Incident::create($request->all())`, n'importe qui pourrait ajouter un champ caché `declare_par_id=1` avec les outils du navigateur et se faire passer pour quelqu'un d'autre.

Deux protections :

1. n'utiliser que `$request->validated()` ;
2. fixer côté serveur ce qui ne dépend pas de l'utilisateur : `'declare_par_id' => $request->user()->id`.

3. Où placer la logique métier ?

Règle	Où ?	Pourquoi
Cause racine obligatoire si résolu	Form Request	C'est une règle de **validation**
Date de rétablissement = maintenant si vide	Contrôleur (méthode privée)	Dépend de l'action de l'utilisateur
Calcul de la durée	Modèle `Incident`	Utilisé partout (liste, détail, rapport)
Référence `INC-2026-00042`	Événement du modèle	Doit exister quelle que soit la façon de créer l'incident

```
// Modèle Incident
public function dureeMinutes(): int
{
    return (int) $this->debut_at->diffInMinutes($this->resolu_at ?? now());
}
```

4. Les dates avec Carbon

Grâce au cast `'debut_at' => 'datetime'`, `$incident->debut_at` est un objet **Carbon** :

```
$incident->debut_at->format('d/m/Y H:i');   // 26/10/2026 08:30
$incident->debut_at->diffForHumans();        // il y a 3 heures
now()->subDays(30);                          // il y a 30 jours
```

Le champ HTML `datetime-local` attend le format `Y-m-d\TH:i` :

```
<input type="datetime-local" name="debut_at"
       value="{{ old('debut_at', $incident->debut_at?->format('Y-m-d\TH:i')) }}">
```

L'opérateur `?->` évite une erreur quand la date est `null` (nouvel incident).

5. L'impression

```
@media print {
    .no-print, nav.navbar { display: none !important; }
}
```

Un simple bouton `onclick="window.print()"` permet d'imprimer ou d'enregistrer en PDF la fiche d'un incident.

Exercices MySQL

1. Lister les incidents ouverts ou en cours, du plus ancien au plus récent.
2. Calculer en SQL la durée en minutes de chaque incident résolu : `TIMESTAMPDIFF(MINUTE, debut_at, resolu_at)`.
3. Trouver les incidents résolus **sans** cause racine (il ne devrait y en avoir aucun : pourquoi ?).
4. Afficher les incidents avec le nom du déclarant **et** le nom du technicien assigné (deux `JOIN` sur `users`). Que se passe-t-il pour un incident non assigné avec un `JOIN` simple ?

Règles métier à implémenter

1. Une référence unique `INC-AAAA-NNNNN` est attribuée automatiquement.
2. Le déclarant est l'utilisateur connecté.
3. Passer en **Résolu** ou **Clos** oblige à renseigner la cause racine et la solution.
4. Si on résout sans date de rétablissement, elle vaut « maintenant ». Si on réouvre, elle est effacée.
5. La date de rétablissement ne peut pas précéder le début.
6. La durée affichée est « début → rétablissement », ou « début → maintenant » si l'incident est ouvert (ex. `1j 4h 12min`).

Travail demandé

1. Créer `IncidentRequest` et `IncidentController` (resource complet).
2. Formulaire en deux colonnes : à gauche description, localisation (site, équipement, liaison), résolution ; à droite sévérité, catégorie, statut, assigné, dates.
3. Page de détail : badges, description, impact, cause et solution, colonne d'informations (site, équipement avec IP, liaison avec ses extrémités, déclarant, assigné, dates, durée).
4. Ajouter `dureeMinutes()` et `dureeLisible()` au modèle.
5. Bouton « Imprimer » avec une feuille de style d'impression.

Livrable

Un incident peut être créé, consulté, modifié et résolu, en respectant les 6 règles.

Auto-évaluation

- [ ] La référence est générée automatiquement et unique
- [ ] Résoudre sans cause racine affiche une erreur explicite en français
- [ ] Réouvrir un incident efface la date de rétablissement
- [ ] J'ai essayé d'injecter `declare_par_id` avec les outils du navigateur : sans effet
- [ ] La fiche incident s'imprime proprement
- [ ] Je sais dire pourquoi chaque règle est à l'endroit où je l'ai mise

Ressources

- Laravel — Form Requests : laravel.com/docs/validation#form-request-validation
- Laravel — Mass assignment : laravel.com/docs/eloquent#mass-assignment
- Carbon : carbon.nesbot.com/docs
- MySQL — TIMESTAMPDIFF : dev.mysql.com/doc/refman/8.4/en/date-and-time-functions.html

Pour aller plus loin

Filtrer la liste des équipements selon le site choisi, avec quelques lignes de JavaScript et un attribut `data-site` sur chaque `<option>`.